Kinematically Constrained EKF for Human Joint-Angle Estimation

from monocular video using YOLO26-Pose on Penn Action: what we are building, why, and how, step by step.

Autonomous Systems mini project · Sulojan · University of Moratuwa · project guide v1 (26 Oct 2026)

The project in three sentences

A deep pose network (YOLO26n-pose) gives 17 body keypoints per video frame, but they **jitter**, sometimes **disappear**, and the bones between them **change length** from frame to frame. We model the person as a **2D stick figure** (root position + segment angles + bone lengths) and track it with an **Extended Kalman Filter**, whose measurement model turns the stick figure into keypoints through sin/cos forward kinematics. The output is a smooth, anatomically consistent **knee and hip angle** per frame, evaluated against Penn Action ground truth and compared with simpler smoothers and a 9× slower pose model.

Why an EKF, and not a plain Kalman filter? (the core argument of the report)

A keypoint's position depends on the segment angles through **sin and cos** (Step 4), so the measurement model $h(x)$ is nonlinear. A linear KF can only smooth each keypoint's x and y independently. It can't hold a bone at a constant length, can't say a knee bends no more than 165°, and never estimates the joint angle itself. With angles and lengths in the state, all three come for free. The EKF handles the nonlinearity by linearising h at the current estimate, using the Jacobian H.

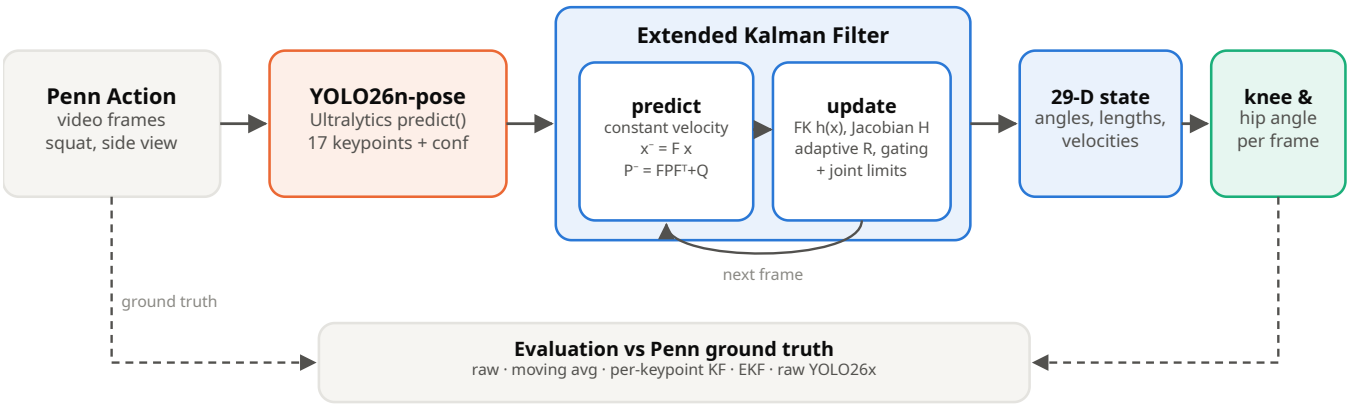


Figure 1. Pipeline at a glance. Only the orange block uses a library (Ultralytics, for `model.predict()` only). Everything in blue is our own NumPy code: the EKF, baselines and metrics.

How to read this guide

Each page is one step, in the order we build it. The top-right label on each step names the code file it lives in. Coloured boxes mean:

Why this design: the argument for the report.

Finding from our own data so far.

Pitfall: where bugs usually hide.

Step 0 The problem, on real frames
Step 1 Data and clip selection
Step 2 Pose detection and caching
Step 3 The skeleton state (29-D)
Step 4 Measurement model h(x)
Step 5 The Jacobian H(x)
Step 6 Process model F, Q
Step 7 One EKF frame, end to end
Step 8 Initialisation by inverse kinematics
Step 9 Baselines and evaluation
Step 10 Experiments E1 to E11
Step 11 Where we are and what's next

STEP 0 The problem, on real frames

motivation

Below is test clip **1663** (squat, side view). Grey is the Penn Action ground-truth skeleton; orange is the raw YOLO26n-pose output on the same frame. Most of the time the network is close; sometimes a joint is clearly off.

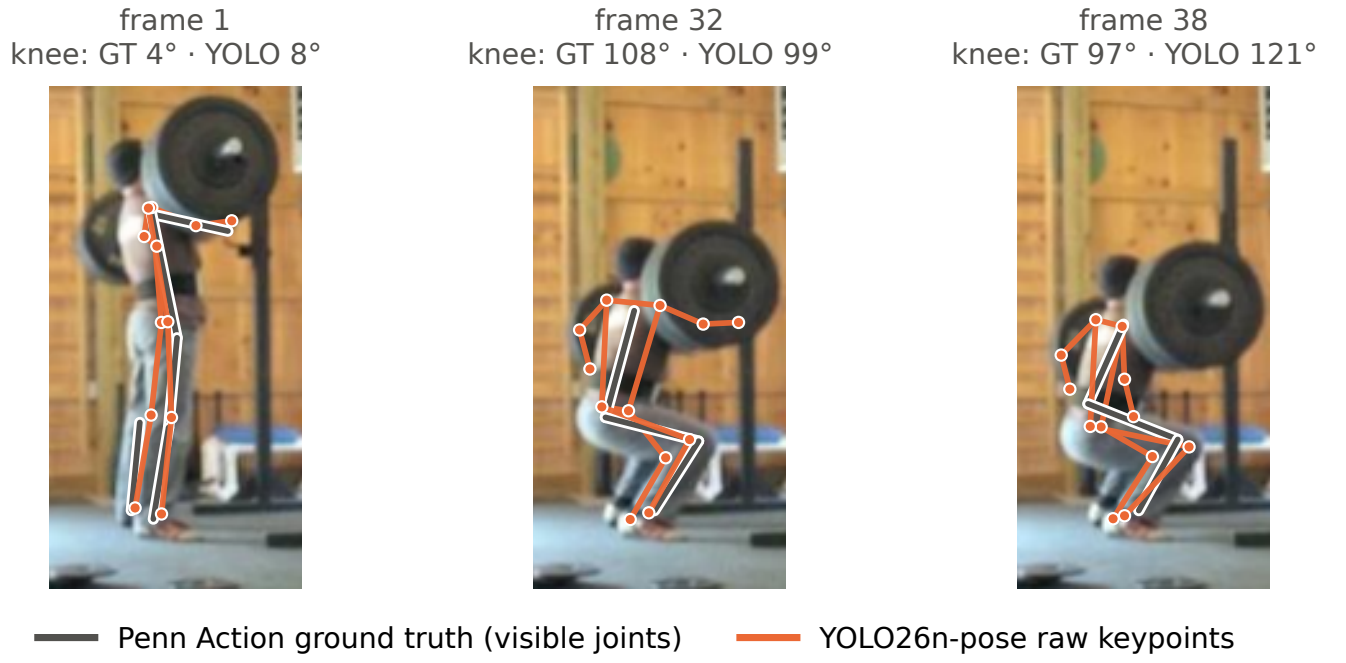


Figure 2. Three frames of clip 1663: standing, deepest squat, and the frame with the largest raw knee-angle error. Ground truth is drawn only for joints Penn marks as visible.

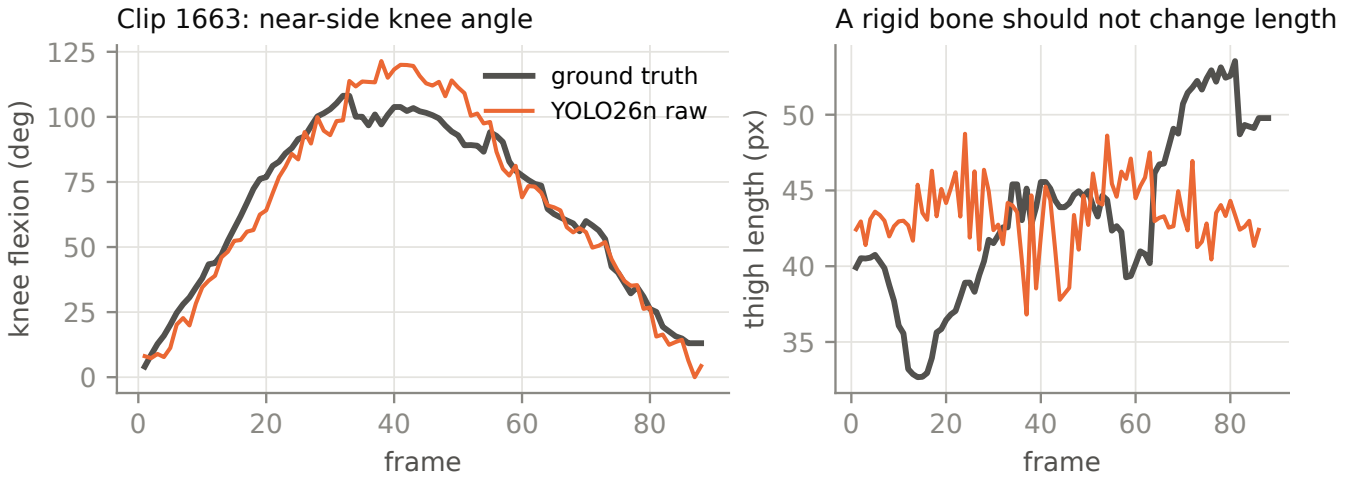


Figure 3. Left: near-side knee flexion from raw YOLO keypoints versus ground truth. Right: the thigh length implied by the raw keypoints. A real thigh is rigid, yet its 2D length jumps around frame to frame. A per-keypoint smoother cannot fix this, because it doesn't know the two keypoints are joined by a bone.

9.6°	**3.9 vs 2.2**	**2.9%**
median knee-angle RMSE, raw YOLO26n, 37 test clips	angle jitter (deg/frame^2), raw YOLO26n vs ground truth	median keypoint error, % of body height (near-side leg)

Preliminary numbers from the cached keypoints, computed with the left/right fix from Step 2. These are the "raw" rows of table E6; the EKF must beat them.

STEP 1 Data and clip selection

`data_io/penn.py` · DONE

Penn Action (Zhang, Zhu & Derpanis, ICCV 2013) has 2,326 short sports and exercise clips with 13 hand-annotated joints per frame and a per-joint *visibility* flag. Penn's joints 1 to 12 are exactly COCO keypoints 5 to 16 (shoulders, elbows, wrists, hips, knees, ankles), so ground truth and YOLO output line up without a lookup table.

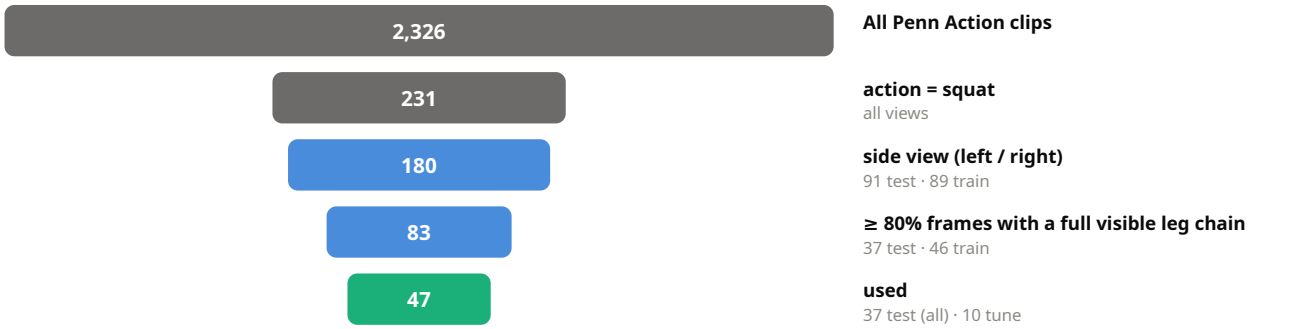


Figure 4. From the whole dataset to the clips we actually use. The visibility rule keeps clips where the camera sees one complete leg chain (shoulder → hip → knee → ankle) in at least 80% of frames. The filter tracks every joint; this rule only makes sure there is ground truth to score the knee and hip against.

Rules that keep the evaluation honest

- **Tune on train, report on test.** Q and R are chosen on the 10 tuning clips (`train = 1`). The 37 test clips (`train = −1`) are used only for final numbers.
- **Ground truth only where visible.** Invisible joints hold junk coordinates such as (0.7, 0.7). An angle is scored only on frames where all of its joints are visible.
- **dt = 1 frame.** Penn doesn't give a frame rate, so noise parameters are in px/frame² and rad/frame².

What a clip looks like

field	shape	meaning
`kp`	(T, 13, 2)	GT joint pixels
`vis`	(T, 13)	True if annotated visible
`bbox`	(T, 4)	person box, for % height errors
`view`	str	left / right / front / back

T ≈ 64 to 163 frames, about 315 × 480 px. Clip 1865's bbox is one row short; the loader pads it.

STEP 2 Pose detection and caching

`pose/run_pose.py` · DONE

Every frame of the 47 clips goes through two pose models once. For each frame we keep **the person with the largest box** (Penn has one main subject; bystanders are smaller and can still score high confidence). Results are cached to `results/keypoints/<clip>_<model>.npz`, so no experiment ever runs the network again, and every method sees **identical keypoints**.

```
kp          (T, 17, 3)   x, y, confidence    zeros + conf 0 if nobody detected → EKF just predicts
box         (T, 4)       chosen person's box
n_persons   (T,)         detections per frame
t_infer_ms  (T,)         network time only (for the edge-AI study, E7)
```

5.5 ms
YOLO26n median inference / frame (RTX 2050)

52 ms
YOLO26x median inference / frame, about 9× slower

2 / 35
frames with no detection (nano / x) over all 47 clips

Finding 1: Penn Action's left and right are mirrored relative to COCO in side views

On every one of the **45 clips** where both legs are visible, swapping Penn's L and R labels matched the YOLO keypoints better, for **both** models: 98% (nano) and 99.7% (x) of the ~6,800 knee and ankle pair comparisons. The other 2 clips never show both legs, so they can't be tested. This is a dataset convention, not detector noise: a person facing left shows their anatomical *left* side to the camera, which Penn labels "R".

Fix: swap Penn's L/R joint labels globally when loading ground truth (and the `side` column with them). Without it, the raw knee and ankle errors on clip 1663 look like about 9% of body height; with it, the median over the test set is 2.9%. Every angle in E5 to E7 depends on this. It's a good "limitation handled" paragraph for the Discussion.

Finding 2: confidence doesn't drop for hidden joints

On clip 1663 the far-side shoulder and hip (invisible in GT) get confidence 0.97 to 0.99. So the confidence-adaptive R (Step 7) helps with blur and partial detections, but will not by itself reject self-occluded far-side limbs. In side views those limbs sit close to the near side, which limits the damage. State this honestly as a limitation.

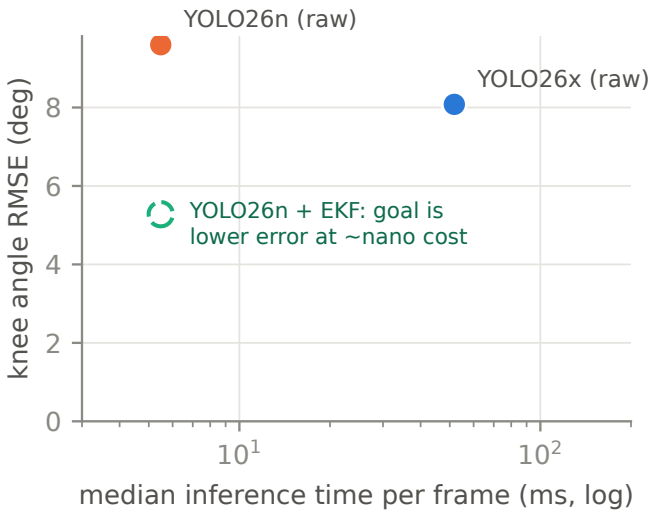


Figure 5. The edge-AI question behind E7, with today's raw numbers: 9× the compute buys only a small accuracy gain. Can nano + EKF reach raw x at nano cost? The dashed point is the target, not a result.

STEP 3 The skeleton state (29-D)

ekf/skeleton.py · IN PROGRESS

Instead of tracking 12 independent keypoints, the filter tracks the **causes** of the keypoints: where the body is, how each segment is rotated, how fast it is rotating, and how long each bone is.

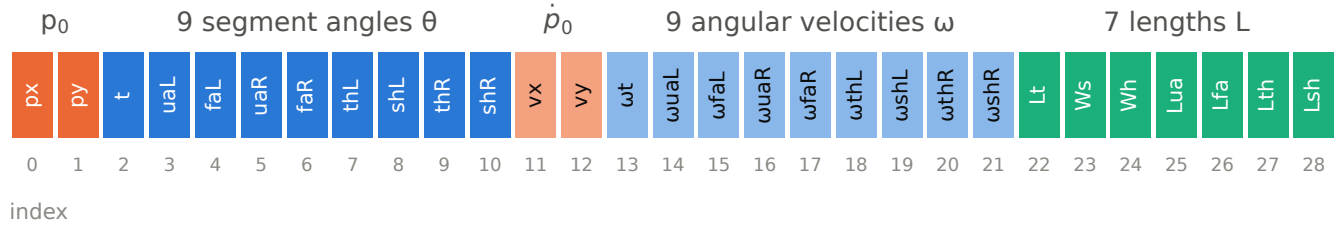


Figure 6. State vector layout. Left and right limbs share one length per segment type. The code uses named index constants (`ANG_THIGH_L = 7`, ...) and never raw numbers.

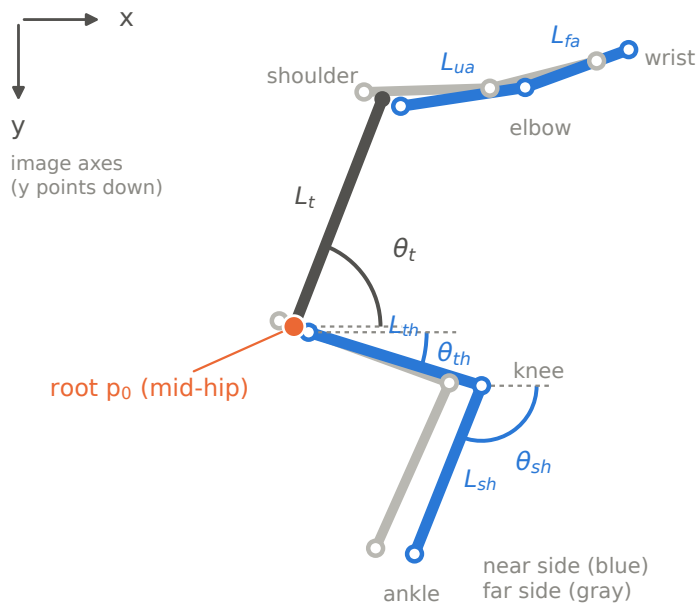


Figure 7. The planar body model in image coordinates. Each angle is measured from the +x axis. Because **y points down**, positive angles turn **clockwise** on screen: an upright torso has $\theta_t \approx -\pi/2$, a hanging limb $\approx +\pi/2$.

Conventions

$$u(\theta) = [\cos\theta, \sin\theta]^T$$
$$u_\perp(\theta) = u(\theta + \pi/2) = [-\sin\theta, \cos\theta]^T$$

- **Absolute angles**, θ = atan2(Δy, Δx) of each segment. Joint angles are differences of these.
- All angles **wrapped** to (−π, π].
- **Widths** W_s, W_h are half shoulder and hip widths, perpendicular to the torso. In a side view they collapse to about 0, so they are clamped to at least 1 px; other lengths are clamped to at least 5 px.
- **Lengths are states**, so the filter calibrates the person's bone lengths online (bone-length self-calibration, plotted in E8).

Pitfall

Forgetting that y is down flips every sign. `test_standing_pose_y_down` checks that shoulder < hip < knee < ankle in y for a standing state.

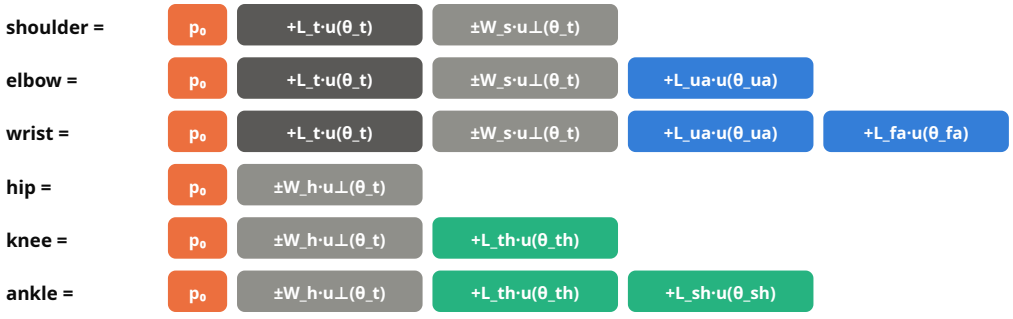
STEP 4 Measurement model h(x): forward kinematics

`ekf/skeleton.py · fk()`

The measurement model predicts where YOLO *should* see each of the 12 keypoints if the state were true. Every keypoint is the root plus a sum of segment vectors along its **kinematic chain**:

$$k_j = p_0 + \Sigma_{i \in \text{chain}(j)}\, s_i\, \ell_i\, d_i(\theta_{a_i}), \quad d_i \in \{\, u, u_\perp \,\}, \quad s_i = \pm 1$$

z = h(x) ∈ ℝ²⁴ (12 keypoints × [x, y])



Sign: + for left, − for right on the width segments only. Same table drives fk() and jacobian().

Figure 8. The segment table. Each keypoint is reached by walking segments from the root; the left/right sign only matters for the width segments. Because *h* and *H* both loop over this one table, the full body costs no more code than a single leg.

Written out

```
mid_sh     = p0 + L_t·u(θ_t)
shoulder_L = mid_sh + W_s·u⊥(θ_t)          shoulder_R = mid_sh − W_s·u⊥(θ_t)
hip_L      = p0     + W_h·u⊥(θ_t)          hip_R      = p0     − W_h·u⊥(θ_t)
elbow_X    = shoulder_X + L_ua·u(θ_ua_X)   wrist_X    = elbow_X + L_fa·u(θ_fa_X)
knee_X     = hip_X      + L_th·u(θ_th_X)   ankle_X    = knee_X  + L_sh·u(θ_sh_X)      (X = L or R)
```

Why this is nonlinear

Angles enter through cos and sin, and they multiply lengths (ℓ cos θ). Doubling θ doesn't double the keypoint shift. This is the property that makes this an **Extended** KF problem. In a side view the widths are about 0, so left and right keypoints nearly coincide; the model still works because the widths are estimated, not assumed.

STEP 5 The Jacobian H(x) = ∂h/∂x

`ekf/skeleton.py · jacobian() · tests/test_jacobian.py`

The EKF needs the local slope of *h* to propagate uncertainty from the state to the keypoints. One observation makes the whole derivation short: **differentiating a direction vector rotates it by +90°**.

$$\mathrm{d}/\mathrm{d}\theta\; u(\theta) = u_{\perp}(\theta), \quad \mathrm{d}/\mathrm{d}\theta\; u_{\perp}(\theta) = -u(\theta) \quad \Rightarrow \quad \mathrm{d}/\mathrm{d}\theta\; d = R_{90} d = [-d_y, d_x]^{\mathrm{T}} \text{ for both}$$

So for every segment *i* on keypoint *j*'s chain:

$$\partial k_j/\partial p_0 = I_2 \qquad \partial k_j/\partial \theta_{a_i} \mathrel{+}= s_i\, \ell_i\, R_{90} d_i \qquad \partial k_j/\partial \ell_i \mathrel{+}= s_i\, d_i \qquad \partial k_j/\partial(\text{velocities}) = 0$$

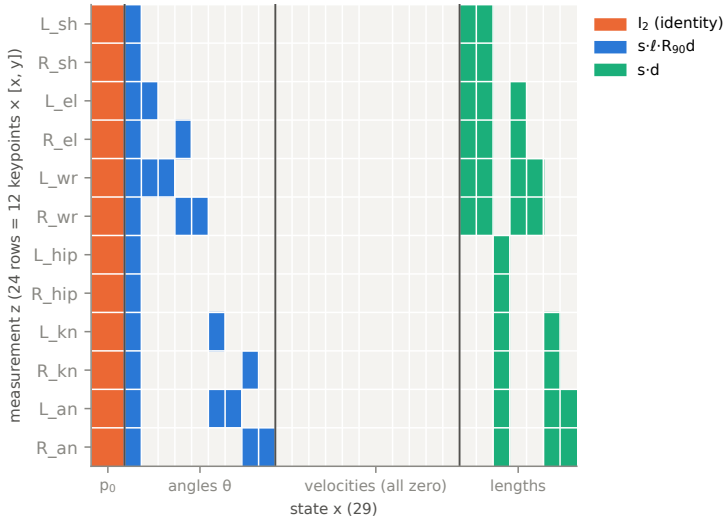


Figure 9. Non-zero pattern of the 24 × 29 Jacobian. Each keypoint (two rows) depends only on the root, the angles and lengths on its own chain, and never on velocities. Wrists depend on 4 angles; hips on just 1.

Pitfall: use += not =

θ_t appears **twice** in each shoulder chain (torso along *u*, half-width along $u_{\perp}$), so its two contributions must add:

$$\partial\ \text{shoulder}_{\mathrm{L}}/\partial\theta_t = L_t\, u_{\perp}(\theta_t) - W_s\, u(\theta_t)$$

How we know it's right

Central finite differences on 20 random states:

$$H_{:,k} \approx [h(x + \varepsilon e_k) - h(x - \varepsilon e_k)] \,/\, 2\varepsilon, \quad \varepsilon = 10^{-6}$$

The error is $O(\varepsilon^2)$, far below the 10^{-5} tolerance. This test **is never dropped**, whatever else gets cut.

STEP 6 Process model: constant velocity

ekf/models.py · F, Q

Between frames we assume each position and angle keeps its velocity, and that any change in velocity is random (**white-noise acceleration**). Bone lengths do a slow random walk, so the filter can refine them but not let them wander quickly.

$$p_0 \leftarrow p_0 + \Delta t \cdot v \quad \theta \leftarrow \theta + \Delta t \cdot \omega$$
$$v, \omega \leftarrow \text{unchanged (+ noise)} \quad L \leftarrow L \text{ (+ noise)}$$
$$x^- = F\,x, \quad P^- = F\,P\,F^{\mathrm{T}} + Q$$

F is the 29 × 29 identity plus Δt = 1 in the 11 (position → velocity) couplings, so prediction is **linear**. Only the update step needs linearising.

For each position or angle and its velocity, Q has the discrete white-noise-acceleration block:

$$Q_{\text{block}} = \sigma^2 \begin{bmatrix} \Delta t^4/4 & \Delta t^3/2 \\ \Delta t^3/2 & \Delta t^2 \end{bmatrix}$$

parameter	start	unit
σ_a root acceleration	1.0	px/frame²
σ_α angular acceleration	0.03	rad/frame²
σ_L length random walk	0.05	px/frame

The trade-off you will tune (E2)

A small σ_α trusts the motion model, giving smooth but laggy angles that cut the bottom of a fast squat. A large σ_α trusts YOLO, giving responsive angles that keep the jitter. The E2 heatmap and the jitter and lag metrics make this trade-off visible.

STEP 7 One EKF frame, end to end

`ekf/ekf.py · predict(), update()`

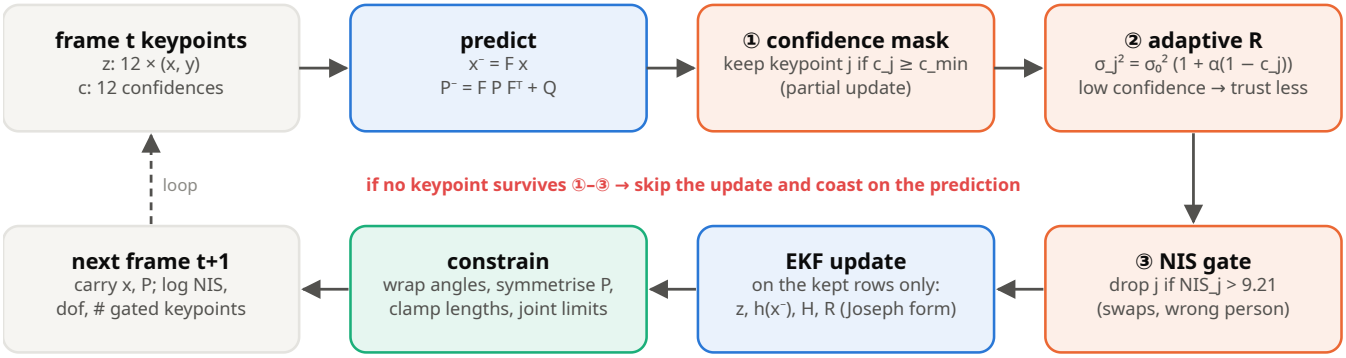


Figure 10. What happens to every video frame. The three "creative" additions, adaptive R, partial updates and NIS gating, are just three ways of choosing **which rows** of *z*, *h*, *H*, *R* to keep, so they share one code path.

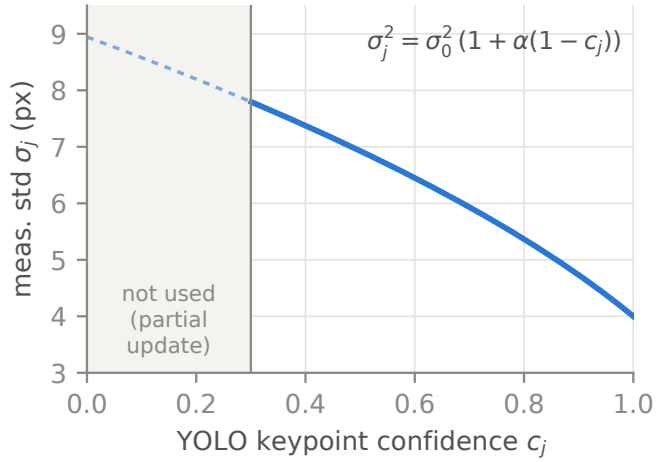


Figure 11. Confidence-adaptive R. Low-confidence keypoints get a larger σ, so they move the state less. Below c_{min} = 0.3 they are dropped entirely (partial update). If no keypoint survives, the filter coasts on prediction.

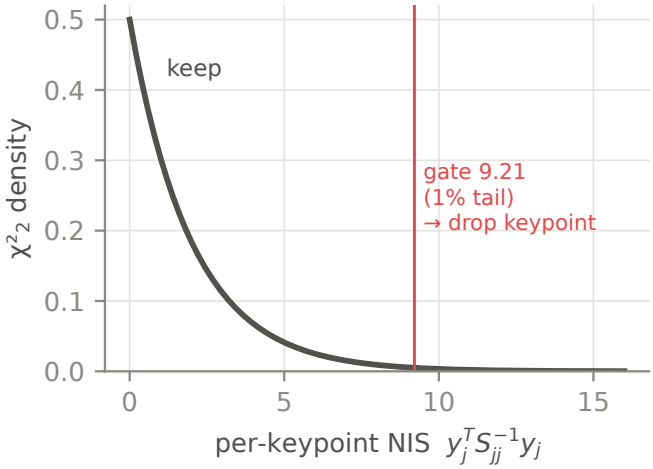


Figure 12. Per-keypoint NIS gating. If a keypoint's normalised innovation exceeds $\chi^2_2(0.99) = 9.21$, it is too surprising given *S*. That usually means a left/right swap or a wrong person, so it is skipped for this frame and counted.

The update equations (use these in the Methodology section)

$$y = z - h(x^-) \quad S = H P^- H^T + R \quad K = P^- H^T S^{-1} \quad \text{(np.linalg.solve, never inv)}$$

$$x = x^- + K y \quad P = (I - KH) P^- (I - KH)^T + K R K^T \quad \text{Joseph form: stays symmetric positive definite}$$

After every update: wrap the angles; symmetrise *P*; clamp the lengths; then apply **joint limits** by projection. If $|\text{wrap}(\theta_{thigh} - \theta_{shank})| > 165°$ the shank angle is moved back onto the limit (160° for elbows), sign-agnostic so left- and right-facing clips both work. In the report, compare this with constrained-KF and truncated-Gaussian approaches.

Logged every frame for consistency checks (E3): NIS of the full update and its degrees of freedom (2 × number of keypoints kept), the number of gated keypoints, and the state and covariance.

STEP 8 Initialisation by inverse kinematics

`ekf/skeleton.py · init_from_keypoints()`

A filter is only as good as its starting point. We read the first usable frame (at least one hip and one shoulder above c_{min}) and invert the forward kinematics geometrically:

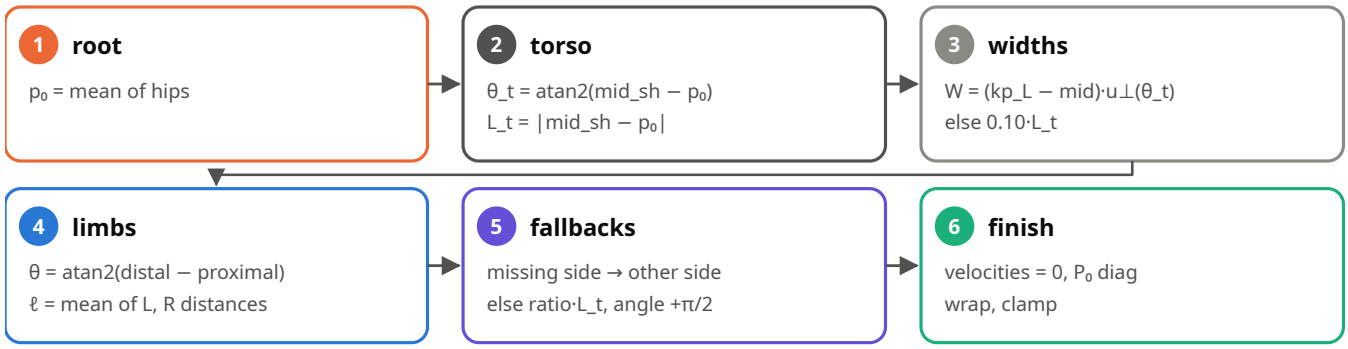


Figure 13. The initialisation recipe, root first, then outward. Missing pieces fall back to the other side, then to anthropometric ratios of torso length (Winter: thigh ≈ shank ≈ 0.82 L_t, upper arm 0.62, forearm 0.49).

state group	initial value	initial σ (P_0)
root p_0	mean of usable hips	10 px
angles θ	atan2 of segment vectors	0.2 rad
root velocity / angular velocity	0	10 px/frame · 0.3 rad/frame
lengths	measured distances (mean of L and R)	5 px

Self-check built into the tests

`test_fk_inverts_init` : generate keypoints from a known state, initialise from them, and check that $h(x_0)$ reproduces the same keypoints. If IK and FK disagree, one of them has a sign or convention bug.

STEP 9 Baselines and evaluation

`baselines/ · eval/metrics.py`

What the EKF is compared against (E6)

method	what it does	what it can't do
Raw YOLO26n	nothing	jitter, missing frames
Causal moving average	mean of the last *N* frames per keypoint	adds lag, ignores occlusion and confidence
Per-keypoint linear KF	constant-velocity KF on each (x, y) independently	no bones: lengths drift, no angles, no limits
One Euro filter (optional)	speed-adaptive low-pass	same as above
EKF (ours)	skeleton state, nonlinear FK model	planar 2D only (see Discussion)
Raw YOLO26x (E7)	9× slower model, no filtering	runtime cost

Fairness: every method uses the same cached YOLO26n keypoints, the same confidence threshold, and the same angle definitions. For non-EKF methods the segment angles are computed from their keypoints.

Angle definitions (identical for GT and all methods)

$$\text{knee} = |\text{wrap}(\theta_{\text{thigh}} - \theta_{\text{shank}})|$$
$$\text{hip} = \pi - |\text{wrap}(\theta_{\text{torso}} - \theta_{\text{thigh}})|$$

Both are 0° standing and grow as you squat. Taking magnitudes makes them independent of which way the person faces.

Metrics

metric	unit
angle RMSE vs GT (visible frames)	deg
keypoint RMSE	% bbox height
jitter = mean |2nd difference|	deg/frame^2
lag = argmax cross-correlation	frames
bone-length CV = std / mean	%
runtime per frame	ms
NEES / NIS vs χ^2 bounds (sim)	consistency

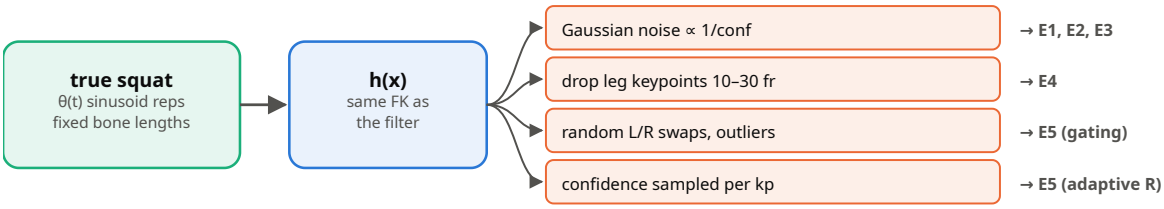
Pitfall: the left/right mirror from Step 2

Apply the global Penn L/R swap inside the ground-truth loader, once, before any metric. Then "near-side leg" means the same leg for GT, YOLO and the EKF.

STEP 10 Experiments: what goes into the Results section

`sim/ · experiments/`

Two kinds of evidence. **Simulation** gives perfect ground truth for the whole state, so we can check the filter is statistically *consistent*. **Real data** shows it actually helps on YOLO output.



Simulation knows the full true state, so NEES (state error) can be checked, not just NIS.

Figure 14. The simulator: a synthetic squat with known angles and lengths, passed through the *same* h(x), then corrupted in the ways YOLO fails. Each corruption maps to one experiment.

#	experiment	data	output	priority
E1	knee angle: truth vs noisy vs EKF with ±2σ band	sim	time-series figure	core
E2	3 × 3 sweep of $\sigma_\alpha \times \sigma_0$	sim (+ tune clips)	RMSE heatmap	core, can shrink to 1-D
E3	NEES and NIS vs χ^2 95% bounds	sim	figure + one-line verdict	never drop
E4	leg keypoints removed for 10 to 30 frames	sim	error and σ growth, then recovery	core
E5	ablation: fixed vs adaptive R, ± gating, ± limits	sim + real test	table	core
E6	raw vs MA vs per-keypoint KF vs EKF	real test (37)	main results table	never drop
E7	nano + EKF vs raw nano vs raw x	real test	table + accuracy-vs-runtime plot	core
E8	bone-length estimates over time	real, 1 to 2 clips	figure	cheap extra
E10	GT / raw / EKF skeleton overlays	real, 1 clip	figure (3 to 4 frames)	core
E11	front squat, jumping jacks failure cases	real	figure or text	optional

What "consistent" means (E3), in one line for the report

If the filter's covariance P is honest, the normalised error $(x - \hat{x})^T P^{-1} (x - \hat{x})$ follows a χ^2 distribution with 29 degrees of freedom: the average NEES should sit near 29 and inside the 95% band. Too high means overconfident (Q or R too small); too low means overly cautious.

STEP 11 Where we are and what's next

status at time of writing

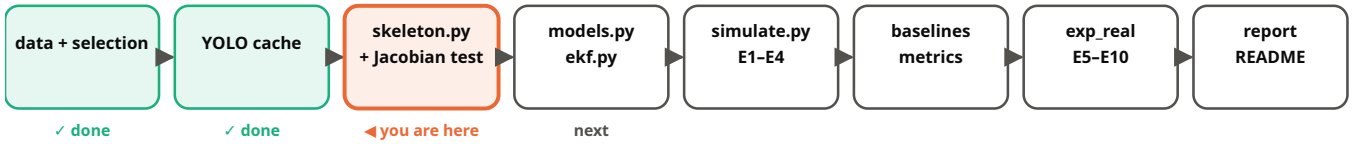


Figure 15. Build order. Each block depends only on blocks to its left, so the order is fixed; what can shrink is the size of each experiment.

file	what's in it	status
`data_io/penn.py`	labels, frames, clip selection	done · add the global L/R mirror
`pose/run_pose.py`	YOLO cache, 94 files	done
`ekf/skeleton.py`	indices, chains, h, H, IK, limits	8 TODOs left
`tests/test_jacobian.py`	finite differences, conventions	1 / 26 passing
`ekf/models.py`, `ekf/ekf.py`	F, Q, R; predict / update / gating / history	next
`sim/simulate.py`, `experiments/exp_sim.py`	synthetic squats; E1 to E4	to do
`baselines/*`, `eval/metrics.py`	MA, per-keypoint KF; all metrics	to do
`experiments/exp_real.py`	E5 to E8, E10	to do
report (Overleaf) + README	8 to 10 pages	to do

Immediate next step

1. Finish the 8 TODOs in `ekf/skeleton.py` plus `numerical_jacobian`, in the order: wrap and helpers → chains → fk → jacobian → constraints → IK → flexion.
2. `python -m pytest tests/test_jacobian.py -v` until 26 / 26 pass.
3. Then `ekf/models.py` and `ekf/ekf.py`.

If time runs short, cut in this order

E11 → E8 → E2 down to a 1-D sweep → One Euro → use only the near-side leg in plots.

Never cut: the Jacobian test, E3 consistency, E6 baseline table.

Report outline (8 to 10 pages): where each step lands

section	content	from steps
1 Introduction	pose for exercise/rehab; noise, jitter, occlusion; 5 contributions: adaptive R, partial updates, bone-length self-calibration, joint limits + NIS gating, edge-AI study	0
2 Methodology	conventions, state, F and Q, FK model, Jacobian derivation, EKF + Joseph form, adaptive R / gating, limits, IK init	3 to 8
3 Implementation	pipeline (Fig. 1), snippets of h(x), H(x), update; Jacobian test; environment	1, 2, 5
4 Results	E1 to E4 (sim), E5 to E8, E10 (real)	10
5 Discussion	Q/R trade-off, NEES/NIS verdict, L/R mirror and overconfident occlusions, 2D planar limits, constant-velocity under fast motion; future work: 3D, UKF, IMM, learned priors	2, 6, 10
6 Conclusion	headline numbers from E6 and E7	10

References: Kalman (1960); Simon, *Optimal State Estimation*; Bar-Shalom et al., *Estimation with Applications to Tracking and Navigation*; Zhang, Zhu & Derpanis, ICCV 2013 (Penn Action); Lin et al., COCO (2014); Ultralytics YOLO26 docs; Winter, *Biomechanics and Motor Control of Human Movement*; Casiez et al., 1€ filter (CHI 2012) if used.